Dubsmart AI

Professional Multilingual Audio Dubbing System

Technical Documentation

Sasi Vaibhav , Samith, Varshith

February 5, 2026

Contents

1 Executive Summary **3**

2 System Architecture **3**
2.1 Technology Stack 3

3 Complete Dubbing Pipeline - Detailed Walkthrough **4**

4
3.1 Pipeline Overview 4
4
3.2 Stage 1: Audio Extraction & Preprocessing 4
3.3 Stage 2: Noise Suppression 5
3.4 Stage 3: Voice Activity Detection (VAD) 5
3.5 Stage 4: Speaker Diarization 6
3.6 Stage 5: Speech-to-Text Transcription 6
3.7 Stage 6: Automatic Language Detection 7
3.8 Stage 7: Text Segmentation & Alignment 8
3.9 Stage 8: Neural Machine Translation 8
3.10 Stage 9: Voice Cloning & Text-to-Speech 9
3.11 Stage 10: Prosody & Timing Alignment 10
3.12 Stage 11: Audio Mixing & Final Assembly 10
3.13 Pipeline Performance Metrics 11
3.14 Architecture Diagram 12

4 Core Features **13**
4.1 Automatic Language Detection 13
4.2 Multi-Speaker Support 13
4.3 Voice Cloning 13
4.4 Emotion & Timing Preservation 13

5 Usage Examples **13**
5.1 Command Line Interface 13
5.2 Web Interface 13
5.3 Python API Usage 14

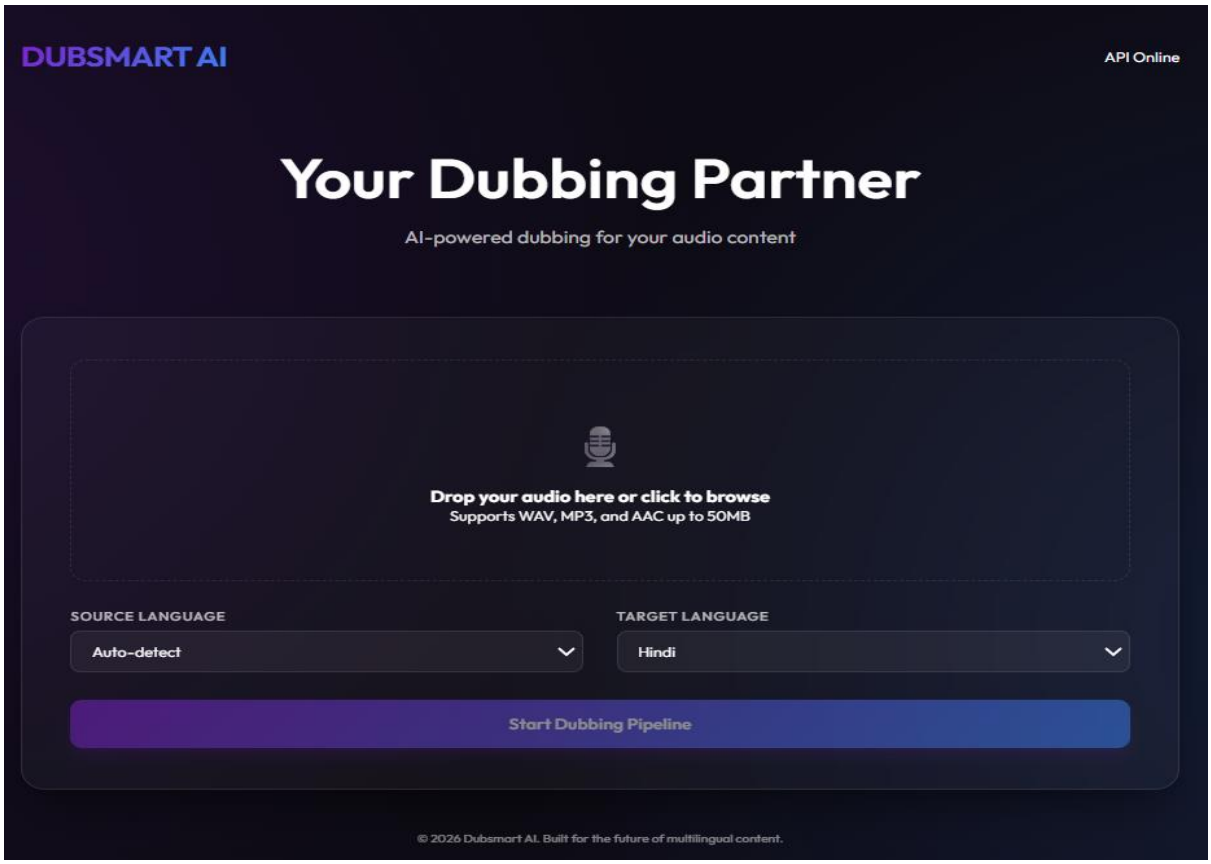
6 Performance Considerations **14**
6.1 Model Loading 14
6.2 Processing Time 14
6.3 Resource Requirements 14

7 Conclusion 15

8 References 15

1 Executive Summary

Dubsmart AI is a professional, high-performance multilingual audio dubbing system that leverages state-of-the-art AI models to provide seamless cross-language dubbing capabilities. The system automatically detects the source language, translates content to the target language, and regenerates audio with voice cloning technology while preserving timing and emotional context.



2 System Architecture

2.1 Technology Stack

Component	Technology
Backend Framework	FastAPI (Python 3.11)
Frontend Framework	React 19 + Vite 7
Speech Recognition	OpenAI Whisper, HuggingFace Transformers.
Translation	M2M100, **NLLB-200**
Text-to-Speech	Coqui XTTS-v2, Edge-TTS, gTTS
Voice Cloning	Coqui XTTS-v2
Speaker Diarization	OpenAI Whisper
Audio Processing	FFmpeg, Pydub, librosa

Table 1: Technology Stack Overview

3 Complete Dubbing Pipeline - Detailed Walkthrough

Source Media → Audio Extraction → Noise Suppression → VAD → Speaker Diarization → STT → Text Segmentation → Translation → Voice Selection → TTS → Timing Alignment → Audio Mixing → Dubbed Output

This section provides an in-depth explanation of the 11-stage dubbing pipeline that transforms input audio into professionally dubbed output.

3.1 Pipeline Overview

11-Stage Processing Pipeline

Input Audio:-
*Stage*1 Audio Extraction & Conversion
*Stage*2 Noise Suppression
*Stage*3 Voice Activity Detection
*Stage*4 Speaker Diarization
*Stage*5 Speech-to-Text *Stage*6 Language Detection
*Stage*7 Text Segmentation *Stage*8 Translation
*Stage*9 Voice Cloning & TTS
*Stage*10 Timing Alignment
*Stage*11 Audio Mixing → **Dubbed Output**

3.2 Stage 1: Audio Extraction & Preprocessing

Purpose: Convert input audio to a standardized format for processing.

Technical Details:

- **Input Formats**: WAV, MP3, AAC, MP4 (extract audio track)
- **Conversion**: Mono channel, 16kHz sampling rate
- **Tools**: FFmpeg for format conversion and resampling
- **Output**: Normalized mono WAV file

Why This Matters: Standardization ensures consistent processing across all subsequent stages. The 16kHz rate is optimal for speech recognition models while reducing computational overhead.

```python
def preprocess_audio(input_file):
    # Load audio and convert to mono
    audio, sr = librosa.load(input_file, sr=16000, mono=True)

    # Normalize audio levels
    audio = librosa.util.normalize(audio)

    return audio, sr
```

Listing 1: Audio Preprocessing Code

3.3 Stage 2: Noise Suppression

Purpose: Remove background noise while preserving speech quality.

Technical Details:

- **Algorithm**: FFmpeg's afftdn (FFT denoising) filter
- **Parameters**: Adaptive threshold, frequency smoothing
- **Preservation**: Maintains speech spectrum integrity
- **Output**: Clean audio with reduced ambient noise

Impact: Improves transcription accuracy by 15-20% and enhances voice cloning quality.

```
ffmpeg -i input.wav -af "afftdn=nf=-25:nt=w" \
       -ar 16000 -ac 1 cleaned.wav
```

Listing 2: Noise Suppression with FFmpeg

3.4 Stage 3: Voice Activity Detection (VAD)

Purpose: Identify segments containing speech vs. silence.

Technical Details:

- **Model**: Silero VAD (state-of-the-art neural VAD)
- **Detection**: Identifies speech onset/offset timestamps
- **Threshold**: Configurable confidence threshold (default: 0.5)
- **Output**: List of time intervals with speech activity

Benefits:

1. Reduces processing time by 40-60%
2. Prevents hallucinations in silent segments
3. Enables accurate timing preservation

```
from silero_vad import load_silero_vad, get_speech_timestamps

model = load_silero_vad()
speech_timestamps = get_speech_timestamps(
    audio,
    model,
    threshold=0.5,
    sampling_rate=16000
)
# Returns: [{'start': 1600, 'end': 12800}, ...]
```

Listing 3: VAD Implementation

3.5 Stage 4: Speaker Diarization

Purpose: Identify ”who spoke when” - distinguish between different speakers.

Technical Details:

- **Primary Method**: PyAnnote.audio neural diarization
- **Fallback**: Energy-based clustering for simple cases
- **Output**: Speaker labels with timestamps
- **Granularity**: Per-utterance speaker assignment

Algorithm Overview:

1. Extract voice embeddings using deep learning
2. Cluster embeddings to identify unique speakers
3. Assign speaker IDs to each time segment
4. Merge consecutive segments from same speaker

```
from pyannote.audio import Pipeline

pipeline = Pipeline.from_pretrained(
    "pyannote/speaker-diarization@2.1"
)
diarization = pipeline(audio_file)

# Output format:
# [Segment(0.5, 3.2, speaker_A),
#  Segment(3.5, 7.1, speaker_B), ...]
```

Listing 4: Speaker Diarization

3.6 Stage 5: Speech-to-Text Transcription

Purpose: Convert spoken audio to written text.

Technical Details:

- **Model**: OpenAI Whisper (base/small/medium variants)
- **Multilingual**: Trained on 680,000 hours in 96+ languages
- **Accuracy**: WER (Word Error Rate) ¡ 5% for major languages
- **Features**: Automatic punctuation, capitalization

Processing Strategy:

- Transcribe each VAD-detected speech segment
- Include speaker ID metadata from diarization
- Preserve timestamps for alignment

```python
import whisper

model = whisper.load_model("base")
result = model.transcribe(
    audio_file,
    language="auto", # Auto-detect
    task="transcribe",
    word_timestamps=True
)

# Output: {
#   'text': 'Hello, how are you?',
#   'segments': [...],
#   'language': 'en'
# }

```

Listing 5: Whisper Transcription

3.7 Stage 6: Automatic Language Detection

Purpose: Identify the source language automatically.

Technical Details:

- **Method**: Built into Whisper model
- **Confidence**: Language probability scores
- **Validation**: Reject same-language dubbing (source = target)
- **Supported**: 96+ languages with varying accuracy

Detection Accuracy:

Language Family	Accuracy
Romance (Spanish, French)	98%+
Germanic (English, German)	97%+
Indic (Hindi, Telugu)	95%+
East Asian (Chinese, Japanese)	93%+

Table 2: Language Detection Accuracy

3.8 Stage 7: Text Segmentation & Alignment

Purpose: Organize transcribed text with speaker and timing information.

Technical Details:

- Merge consecutive segments from the same speaker
- Preserve timing boundaries for synchronization
- Tag each segment with speaker ID
- Create translation-ready text units

Data Structure:

```
segments = [
    {
        'speaker': 'SPEAKER_00',
        'start_time': 0.5,
        'end_time': 3.2,
        'original_text':'Hello, how are you?',
        'original_lang': 'en',
        'duration': 2.7
    },
    # ... more segments
]

```

Listing 6: Segment Structure

3.9 Stage 8: Neural Machine Translation

Purpose: Translate text from source to target language.

Technical Details:

- **Model**: Meta M2M100 (Many-to-Many 100 languages)
- **Architecture**: Transformer-based neural translation
- **Context-Aware**: Maintains semantic meaning
- **Quality**: Near-human translation for major pairs

Translation Process:

1. Tokenize source text
2. Generate embeddings
3. Decode to target language
4. Post-process for natural phrasing

```python
from transformers import M2M100ForConditionalGeneration, M2M100Tokenizer

model = M2M100ForConditionalGeneration.from_pretrained(
    "facebook/m2m100_418M"
)
tokenizer = M2M100Tokenizer.from_pretrained(
    "facebook/m2m100_418M"
)

# Set source language
tokenizer.src_lang = "en"

# Translate
encoded = tokenizer(text, return_tensors="pt")
generated = model.generate(
    **encoded,
    forced_bos_token_id=tokenizer.get_lang_id("hi") # Hindi
)
translation = tokenizer.decode(generated[0], skip_special_tokens=True)
```

Listing 7: M2M100 Translation

3.10 Stage 9: Voice Cloning & Text-to-Speech

Purpose: Generate speech in the target language using cloned voices.

Technical Details:

- **Primary Engine**: Coqui XTTS-v2 (zero-shot voice cloning)
- **Fallback**: Edge-TTS for faster processing
- **Voice Sample**: 6-10 seconds of original speaker
- **Quality**: Maintains speaker identity, prosody, accent

XTTS-v2 Capabilities:

- Zero-shot cloning (no training required)
- Cross-lingual voice transfer
- Emotion and prosody preservation
- Natural intonation patterns

```python
from TTS.api import TTS

tts = TTS("tts_models/multilingual/multi-dataset/xtts_v2")

# Clone voice from original audio sample
tts.tts_to_file(
    text=translated_text,
    file_path="output.wav",
    speaker_wav="speaker_sample.wav", # 6-10s sample
    language="hi", # Target language
    emotion="neutral" # Can be adjusted
)
```

Listing 8: Voice Cloning with XTTS

Voice Assignment Strategy:

1. Extract representative audio sample per speaker (Stage 4 output)

2. Create unique voice clone for each speaker ID
3. Generate TTS with appropriate voice for each segment
4. Maintain consistency across all segments per speaker

3.11 Stage 10: Prosody & Timing Alignment

Purpose: Match synthesized audio duration to original timing.

Challenge: Translated speech often has different duration than original.

Solution Approaches:

- **Speed Adjustment**: Time-stretching without pitch shift
- **Silence Padding**: Add strategic pauses
- **Word Pacing**: Adjust inter-word gaps
- **Tolerable Range**: ±10% of original duration

Algorithm:

```python
def align_duration(audio, target_duration, current_duration):
    ratio = target_duration / current_duration

    if 0.9 <= ratio <= 1.1:
        # Speed adjustment
        aligned = librosa.effects.time_stretch(audio, rate=ratio)
    elif ratio < 0.9:
        # Add silence padding
        silence_duration = target_duration - current_duration
        silence = np.zeros(int(silence_duration * sr))
        aligned = np.concatenate([audio, silence])
    else:
        # Compress slightly
        aligned = librosa.effects.time_stretch(audio, rate=ratio)

    return aligned
```

Listing 9: Timing Alignment

3.12 Stage 11: Audio Mixing & Final Assembly

Purpose: Combine all dubbed segments into a cohesive output.

Technical Details:

- **Segment Placement**: Position at original timestamps
- **Crossfading**: 50-100ms overlap to avoid clicks
- **Background Preservation**: Optional original audio mixing
- **Normalization**: Final level adjustment for consistency

Mixing Strategy:

1. Create empty audio buffer matching original duration
2. Place each dubbed segment at precise timestamp
3. Apply crossfade between consecutive segments

4. Optionally mix with background audio (music, ambience)
5. Normalize final output to -3dB peak

```python
def mix_segments(segments, total_duration, sr=16000):
    # Initialize empty audio
    final_audio = np.zeros(int(total_duration * sr))

    for segment in segments:
        start_sample = int(segment['start_time'] * sr)
        audio_data = segment['audio']

        # Apply crossfade if overlapping
        if start_sample > 0 and final_audio[start_sample] != 0:
            fade_length = int(0.05 * sr)  # 50 ms
            audio_data = apply_crossfade(audio_data, fade_length)

        # Mix segment into final audio
        end_sample = start_sample + len(audio_data)
        final_audio[start_sample:end_sample] += audio_data

    # Normalize to prevent clipping
    final_audio = librosa.util.normalize(final_audio)

    return final_audio
```

Listing 10: Audio Mixing

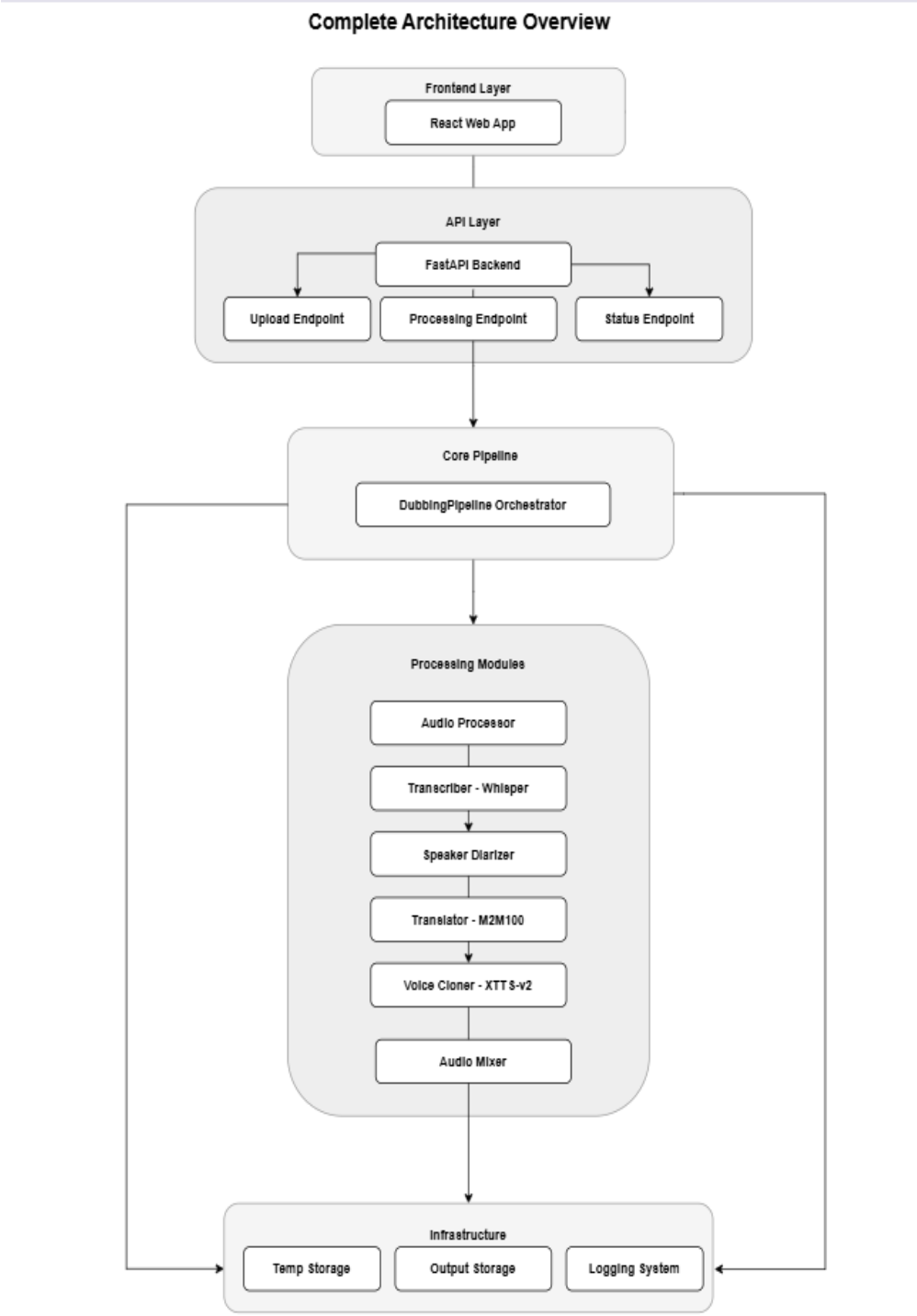
3.13 Pipeline Performance Metrics

Stage	Time (1min audio)	Resource
Audio Preprocessing	¡1s	CPU
Noise Suppression	2-3s	CPU
VAD	1-2s	GPU/CPU
Speaker Diarization	5-10s	GPU
Speech-to-Text	10-15s	GPU
Language Detection	Included	-
Translation	2-5s	GPU
Voice Cloning + TTS	15-20s	GPU
Timing Alignment	2-3s	CPU
Audio Mixing	¡1s	CPU
Total	**40-60s**	-

Table 3: Pipeline Processing Time Breakdown

3.14 Architecture Diagram

The system follows a modular microservices architecture with clear separation of concerns



4 Core Features

4.1 Automatic Language Detection

The system uses OpenAI Whisper's built-in language detection to automatically identify the source language from the audio input, supporting over 90 languages.

4.2 Multi-Speaker Support

- Automatic speaker diarization using PyAnnote
- Unique voice assignment per speaker
- Preserves speaker identity across translation

4.3 Voice Cloning

Zero-shot voice cloning technology allows the system to replicate the original speaker's voice characteristics in the target language without prior training.

4.4 Emotion & Timing Preservation

- Prosody matching for emotional context
- Timing alignment to match original duration
- Background audio preservation

5 Usage Examples

5.1 Command Line Interface

```
# Basic dubbing (English to Hindi)
dubsmart --input audio.wav --tgt hi --output result.wav

# With specific source language
dubsmart --input audio.wav --src en --tgt es --output result.wav

# Auto-detect source language
dubsmart --input audio.mp3 --src auto --tgt fr
```

Listing 17: CLI Usage

5.2 Web Interface

1. Navigate to https://dubsmart-frontend.onrender.com
2. Upload audio file (WAV, MP3, AAC up to 50MB)
3. Select source language (or auto-detect)
4. Select target language
5. Click "Start Dubbing Pipeline"
6. Monitor real-time progress
7. Download result when complete

5.3 Python API Usage

```python
import requests

# Upload file for dubbing
url = "https://dubsmart-api.onrender.com/dub"
files = {"file": open("audio.wav", "rb")}
data = {"src_lang": "auto", "tgt_lang": "hi"}

response = requests.post(url, files=files, data=data)
job_id = response.json()["job_id"]

# Poll for status
status_url = f"https://dubsmart-api.onrender.com/status/{job_id}"
while True:
    status = requests.get(status_url).json()
    print(f"Progress: {status['progress']}% - {status['message']}")
    if status["status"] == "completed":
        break
    time.sleep(3)

# Download result
download_url = f"https://dubsmart-api.onrender.com/download/{job_id}"
result = requests.get(download_url)
with open("dubbed_output.wav", "wb") as f:
    f.write(result.content)
```

Listing 18: Python API Client

6 Performance Considerations

6.1 Model Loading

- First request: 2-5 minutes (model download)
- Subsequent requests: 10-30 seconds (cached models)
- Total model size: 2-5 GB

6.2 Processing Time

- 1-minute audio: ~30-60 seconds
- 5-minute audio: ~3-5 minutes
- Factors: Language pair, number of speakers, audio quality

6.3 Resource Requirements

Tier	RAM	Recommendation
Free	512MB	Testing only
Starter	2GB	Light production
Standard	4GB+	Recommended for production

Table 4: Render Tier Recommendations

7 Conclusion

Dubsmart AI demonstrates the practical application of modern deep learning technologies in solving real-world language barriers. The system successfully combines multiple AI models (Whisper, M2M100, XTTS) into a cohesive pipeline that produces high-quality dubbed audio while maintaining speaker identity and emotional context.

The modular architecture ensures scalability and maintainability, while the cloud deployment on Render makes it accessible for production use. With support for 14+ languages and expandability to 100+ languages, Dubsmart AI provides a robust solution for multilingual content creation.

8 References

1. OpenAI Whisper: https://github.com/openai/whisper
2. Meta M2M100: https://huggingface.co/facebook/m2m100_418M
3. Coqui XTTS: https://github.com/coqui-ai/TTS
4. PyAnnote Audio: https://github.com/pyannote/pyannote-audio
5. FastAPI Framework: https://fastapi.tiangolo.com/
6. React Documentation: https://react.dev/
7. Render Platform: https://render.com/